**relatorio tecnico**

# conflitos de dados e de controle no pipeline

avaliacao pratica m2 - etapa 2 de 2
organizacao de computadores | univali

**integrantes**

Jose Gabriel Santos Gomes

Matheus Pompeo Dias

Nathan Gustavo Reichert

## objetivo

estender o decodificador da pratica m1 para que, a partir do mesmo arquivo de memoria de instrucoes, o programa detecte os conflitos de dados e de controle de um pipeline de cinco estagios sem nenhuma protecao de hardware e produza duas versoes corrigidas do codigo: uma somente com nops e outra com reordenacao de instrucoes e nops. os desvios sao recalculados e os arquivos de saida podem ser lidos novamente pelo decodificador da m1.

### premissas do modelo

pipeline if, id, ex, mem, wb, sem adiantamento, deteccao de conflitos, descarte ou predicao. o banco de registradores e escrito na primeira metade do ciclo e lido na segunda, de modo que um consumidor a distancia **d** do produtor tem conflito quando d = 1 ou d = 2 e precisa de **3 - d** nops. desvios sao resolvidos no id e o slot de atraso sempre executa. o registrador zero nunca gera conflito, acessos com o mesmo registrador-base e deslocamentos diferentes sao enderecos distintos e **nop** e **addi zero, zero, 0** (0x00000013).

### ideia central: fluxo de execucao

o programa e tratado como uma sequencia de posicoes e a pergunta e quem executa logo depois de cada uma. normalmente e a proxima. depois de um desvio ou salto vem sempre o slot de atraso e so entao o fluxo se divide entre o alvo e a instrucao seguinte. com essa relacao, a distancia entre produtor e consumidor e calculada por qualquer caminho, inclusive atravessando desvios, e a mesma funcao serve para detectar os conflitos do original e para verificar o resultado corrigido.

| entrada | etapas | saida |
|---|---|---|
| rom em hex/bin (formato da m1) | parser, decoder (m1), analise de fluxo, nops, reordenacao, recalculo de desvios | dois programas corrigidos (.hex/.bin), duas listagens (.lst) e relatorio de custo no console |

# 1. requisitos atendidos pelo projeto

a tabela relaciona cada requisito do enunciado com o que foi implementado e onde.

| req. | o que pede | implementacao no projeto |
|---|---|---|
| r1<br>1,5 | detectar pares produtor/consumidor que violam a distancia minima, com enderecos, registrador e distancia. somente leitura apos escrita; respeitar os campos de cada formato; produtor mais recente. | findHazards em src/hazards.cpp anda 1 e 2 posicoes para tras pelos predecessores no fluxo de execucao. usa readsRegisters/writesRegister da m1, que deixam vazios os campos que o formato nao tem (s e b nao escrevem) e omitem x0. |
| r2<br>1,0 | identificar desvios e o ocupante do slot, formar blocos basicos e detectar conflitos de dados que atravessam o desvio. | makeContext cria os blocos (alvo inicia, desvio encerra). analyze lista desvio e slot. conflitos cujo caminho sai de um slot sao marcados crossesBranch e impressos em secao propria. |
| r3<br>1,5 | menor numero de nops, sem mudar a ordem, sem conflito de dados restante. | insertDataNops corrige o primeiro consumidor em conflito, antes dele, e recalcula ate zerar. o alvo do desvio passa a apontar para o primeiro nop. se o consumidor e o slot, os nops entram antes do desvio. |
| r4<br>1,0 | preencher o slot de atraso com nop. | buildItems insere um nop depois de cada desvio ou salto (Tag::NopSlot), pois o original supoe que a proxima instrucao nao executa. |
| r5<br>2,0 | reordenar dentro do bloco, preencher o slot com instrucao util e reverificar o inicio dos dois caminhos. | Scheduler: busca exata com memoizacao dentro do bloco (guloso acima de 14 instrucoes). slotCandidates e a escolha por bloco, testando cada candidata contra o programa inteiro. |
| r6<br>1,5 | recalcular deslocamentos de desvio e salto e reescrever em linguagem de maquina. | finalize calcula alvo novo menos posicao nova e withBranchOffset reempacota os bits embaralhados de b e j, com verificacao de alcance. |
| r7<br>1,0 | dois arquivos no formato da entrada, legiveis pelo decodificador da m1, e listagem com marcacoes. | writeOutputs em src/main_m2.cpp gera _nops e _reordenado (.hex ou .bin) e .lst com endereco, codigo, instrucao e marcacao. |
| r8<br>0,5 | custo de cada tecnica e comparacao. | printCost mostra original, final, acrescimo e percentual (virgula decimal) e compara as duas solucoes. |

## distribuicao da nota no enunciado

| criterio | pontos | criterio | pontos |
|---|---|---|---|
| r1 - dados | 1,5 | r5 - reordenacao | 2,0 |
| r2 - controle | 1,0 | r6 - enderecos | 1,5 |
| r3 - nops de dados | 1,5 | r7 - arquivos de saida | 1,0 |
| r4 - nops de controle | 1,0 | r8 - relatorio de custo | 0,5 |
| | | total | 10,0 |

a defesa e individual e pode ajustar a nota de cada integrante.

# 2. algoritmo

## estrutura de dados

cada posicao do programa e um **Item**: a instrucao decodificada, o indice no original (ou -1 para nop inserido), o bloco basico, o bloco de destino quando e desvio e uma etiqueta (original, reordenada, movida para o slot, nop de dados, nop de slot). o mesmo tipo representa a entrada e as saidas. o alvo de um desvio e guardado como bloco, nao como endereco, e so vira numero na etapa final.

## deteccao (r1 e r2)

para cada instrucao e cada registrador que ela le, procuram-se os produtores em 1 e 2 passos para tras. se a instrucao imediatamente anterior no caminho escreve o registrador, a distancia e 1. caso contrario, olha-se a anterior a ela e a distancia e 2. o produtor mais recente encerra a busca naquele caminho. escrita apos leitura e escrita apos escrita nao entram.

## nops de dados (r3)

corrige-se o consumidor mais cedo no programa, inserindo antes dele o maior numero de nops que seus conflitos pedem, e a analise e refeita. como os nops ficam antes do consumidor, valem para todos os caminhos que chegam a ele. o recalculo a cada insercao evita contar duas vezes nops que beneficiam varios consumidores.

## reordenacao (r5)

dentro de cada bloco, a dependencia entre duas instrucoes existe quando ha leitura apos escrita, escrita apos leitura, escrita apos escrita, ordem entre acessos a memoria que podem se sobrepor, ou uma delas e barreira (ecall, csr, palavra invalida). acessos com o mesmo registrador-base, deslocamentos diferentes e base inalterada entre eles sao independentes. o desvio depende de todas as instrucoes do bloco.

a ordem e escolhida por busca exata: o estado e o conjunto de instrucoes ja emitidas mais os registradores escritos pelas duas ultimas. o custo de emitir uma instrucao e 2 nops se ela le o que a ultima escreve e 1 se le o que a penultima escreve. empates ficam com a ordem original.

uma instrucao pode ocupar o slot de atraso se vem de antes do desvio e nenhuma outra instrucao do bloco, nem o proprio desvio, depende dela. preencher o slot costuma criar um conflito novo no inicio dos caminhos, entao cada candidata e avaliada contra o programa inteiro, ja com os nops de dados, e o slot com nop so e mantido se for estritamente melhor. a escolha e repetida bloco a bloco ate estabilizar. chamadas (jal que grava o endereco de retorno) e retornos (jalr) ficam sempre com nop no slot: o retorno cai no proprio slot da chamada, que executa de novo, e uma instrucao movida para la rodaria duas vezes.

## recalculo de enderecos (r6)

com a sequencia final fixada, o deslocamento de cada desvio e salto e: endereco novo do alvo menos endereco novo da instrucao. os bits sao reempacotados nas posicoes embaralhadas do formato b (imm[12|10:5|4:1|11]) ou j (imm[20|10:1|11|19:12]) e a palavra e decodificada de novo. um desvio para logo depois da ultima instrucao acompanha o novo fim do programa. deslocamento fora do alcance (13 bits em b, 21 bits em j) e erro.

# 3. validacao e resultados

## verificacoes automaticas (tests/test_hazards.cpp)

| verificacao | como funciona | resultado |
|---|---|---|
| equivalencia de comportamento | emulador rv32i com duas maquinas: ideal (sem conflito nem slot) e pipeline do enunciado (leitura defasada, slot). o estado final do original na maquina ideal deve ser igual ao de cada saida no pipeline. | 1500 programas aleatorios com lacos, desvios para frente e jal: todos iguais. 1306 deles mudam de resultado sem a correcao. |
| sem conflitos restantes | a deteccao e executada de novo sobre o resultado. | 0 em todos os casos |
| otimalidade | busca exaustiva independente de todas as permutacoes validas em trechos retos curtos. | 400 trechos: tamanho sempre igual ao minimo |
| codificacao de desvios | reescreve deslocamentos e decodifica de volta com o decodificador da m1. | todos os valores e registradores preservados |
| mutacao | desligar a regra de escrita apos leitura na reordenacao. | o teste falha (comportamento muda) |

## casos de teste (tests/data)

| caso | o que cobre | original | so nops | reordenado |
|---|---|---|---|---|
| cadeia | cadeia de dependencias sem instrucao independente | 5 | 13 | 13 |
| sem_nops | reordenacao elimina todos os nops | 9 | 15 | 9 |
| laco | laco com desvio para tras | 8 | 13 | 9 |
| frente | desvio para frente | 7 | 14 | 14 |
| enunciado | laco no estilo do exemplo do enunciado | 9 | 17 | 14 |

em cadeia, cada consumidor usa o resultado da instrucao anterior, nao ha instrucao independente para preencher o espaco e a reordenacao nao ajuda: o resultado e igual ao de somente nops (4 consumidores com 2 nops cada). em sem_nops, intercalar os tres pares produtor/consumidor deixa todos a distancia 3 e nenhum nop e necessario.

## programas reais de arquitetura i (tests/data/arq1)

| programa | original | so nops | reordenado | conflitos |
|---|---|---|---|---|
| m1_programa_01 (registro de presenca) | 37 | 62 | 51 | 12 |
| m1_programa_02 (multiplicacao) | 31 | 43 | 41 | 5 |
| m2_programa_01 (resto da divisao) | 32 | 44 | 43 | 5 |
| m2_programa_02 (numero primo, chamadas) | 45 | 72 | 66 | 10 |
| m2_programa_03 (soma de digitos, recursao) | 43 | 66 | 55 | 6 |
| fibonacci | 29 | 40 | 35 | 5 |
| mdc | 32 | 45 | 39 | 7 |
| soma | 19 | 25 | 22 | 3 |
| vetor | 13 | 27 | 20 | 7 |

os programas foram montados com tests/asm.py (mesmo mapa de memoria do rars) e executados no emulador com entradas simuladas. para cada um, o texto impresso e os registradores (menos ra, que guarda enderecos de retorno) e a area de dados sao iguais aos do original, incluindo as chamadas e a recursao.

## exemplo de saida no console

```
CUSTO
  original ..........   9 instrucoes
  somente nops ......   17 instrucoes  (+8, +88,9%)
  nops + reordenacao    14 instrucoes  (+5, +55,6%)
```

```
Comparacao: a reordenacao evitou 3 de 8 nops (37,5%).
```

## limitacoes

jalr nao tem alvo conhecido: o slot e tratado, o destino nao. auipc e relativo ao pc e muda quando ha insercao de nops (o programa avisa). acessos de tamanhos diferentes que se sobrepoem com a mesma base e deslocamentos diferentes seguem a premissa 6 do enunciado. jal e jalr tem slot de atraso como os desvios condicionais. ecall e tratado pelo ambiente, em ordem de programa, e nao entra como leitura ou escrita de registradores, como no decodificador da m1.

## como executar

```
cmake -S . -B build && cmake --build build
build/riscv-hazards programa.hex -b 0x00400000 -o saida
build/test_hazards
```